

Performance Evaluation of Host Aware Shingled Magnetic Recording (HA-SMR) Drives

Fenggang Wu, Ziqi Fan, Ming-Chang Yang, Baoquan Zhang, Xiongzi Ge, and David H.C. Du, *Fellow, IEEE*

Abstract—Shingled Magnetic Recording (SMR) drives can benefit large-scale storage systems by reducing the Total Cost of Ownership (TCO) of dealing with explosive data growth. Among all existing SMR models, Host Aware SMR (HA-SMR) looks the most promising for its backward compatibility with legacy I/O stacks and its ability to use new SMR-specific APIs to support host I/O stack optimization. Building storage systems using HA-SMR drives calls for a deep understanding of the drive's performance characteristics. To accomplish this, we conduct in-depth performance evaluations on HA-SMR drives with a special emphasis on the performance implications of the SMR-specific APIs and how these drives can be deployed in large storage systems. We discover both favorable and adverse effects of using HA-SMR drives under various workloads. We also investigate the drive's performance under legacy production environments using real-world enterprise traces. Finally, we propose a novel host-controlled buffer that can help to reduce the severity of the decline in HA-SMR performance under our discovered unfavorable I/O access patterns. Without a detailed comprehensive design, we show the potential of the host-controlled buffer by a case study.

Index Terms—Host Aware SMR, characterization, system implication, H-Buffer

1 INTRODUCTION

As perpendicular magnetic recording (PMR) [1] technology used by the conventional HDDs is reaching its areal density limit, other new recording technologies such as Heat-Assisted Magnetic Recording (HAMR) [2], [3], Bit-Patterned Media (BPM) [4] and Shingled Magnetic Recording (SMR) [5], [6], [7] are investigated to overcome this barrier. While both HAMR and BPM are not commercialized yet due to certain technical difficulties, SMR drives have already been available recently to general customers [8].

SMR drives ordinarily have multiple individual shingled *bands* separated by guard tracks to increase flexibility. According to the T10 [9] and T13 [10] standards, the physical SMR bands are abstracted as logical *zones* representing consecutive non-overlapping LBA ranges. Each of the zones is designed to be written in a log structured manner, and is associated with a *write pointer* indicating the location in the zone where the next write should target.

There exists three types of SMR drives: *drive managed*, *host aware*, and *host managed* [11]. Drive Managed SMR (DM-SMR) drives buffer the updating I/O traffic in a *media*

cache (or *persistent cache*, one portion of the disk media) and later migrate those data blocks back to the intended locations (LBAs in the targeted zones) by a *media cache cleaning* process. This media cache as well as the cleaning operations are hidden from the host, therefore, DM-SMR drives interact with the host in the same way as conventional non-SMR drives do [11], [12], [13]. Host Aware SMR (HA-SMR) drives also use the media cache to buffer the updating I/O traffic. Specifically *non-sequential write* requests (i.e., write requests not beginning at the write pointer of a zone) will be redirected to media cache [9], [10]. Furthermore, HA-SMR drives expose internal layout information, such as the number of *zones*, the starting LBA, the write pointer, and various flags of each zone, to the host via *zoned block APIs* [9], [10], [14]. Host Managed SMR (HM-SMR) drives expose the same zoned block APIs as HA-SMR drives do, but they do not accept non-sequential writes and hence do not need the persistent cache component [9], [10], [14].

In this paper, we choose HA-SMR drives as our research subject because we believe HA-SMR is expected to be more popular than the other two models due to the following two reasons. Compared with HM-SMR model, HA-SMR is more backward compatible with legacy software, so SMR drive users can still benefit from the lower TCO without revamping their applications or I/O stacks. Compared with DM-SMR model, HA-SMR provides the opportunity of zone-aware software optimization for a higher and more predictable performance by using zoned block APIs which can expose detailed zone information to the host.

There are fewer existing studies that addresses SMR performance characterization. The closest one-Skylight [12], [13] focuses on uncovering the internal structural information of the “black box” DM-SMR model. A thorough investigation of HA-SMR drives is still missing, especially the

- F. Wu is with the Department of Computer Science and Engineering, University of Minnesota, Twin Cities, Minneapolis, MN 55455. E-mail: wuxx0835@umn.edu.
- Z. Fan, B. Zhang, X. Ge, and D.H.C. Du are with the Department of Computer Science and Engineering, University of Minnesota, Minneapolis, MN 55455. E-mail: {fanxx234, zhan4281}@umn.edu, {xiongzi, du}@cs.umn.edu.
- M.-C. Yang is with the Graduate Institute of Networking and Multimedia, National Taiwan University, Taipei 10617, Taiwan. E-mail: mcyang@iis.sinica.edu.tw.

Manuscript received 5 Jan. 2017; revised 26 Apr. 2017; accepted 21 May 2017. Date of publication 6 June 2017; date of current version 16 Oct. 2017.

(Corresponding author: Fenggang Wu.)

Recommended for acceptance by Z. Shao.

For information on obtaining reprints of this article, please send e-mail to: reprints@ieee.org, and reference the Digital Object Identifier below.

Digital Object Identifier no. 10.1109/TC.2017.2713360

performance implications of the HA-SMR model due to its unique zoned block APIs.

We carry out an in-depth performance evaluation on several HA-SMR sample drives with a special emphasis on the performance characterization regarding the newly-introduced zoned block APIs. For example, we study the performance impact of the zone sequential writes and non-sequential writes regarding the number of zones that are in the writing working set (open zone and non-sequential zone issues). Besides, as media cache cleaning efficiency is the key factor to determine the non-sequential write performance, we investigate how different workloads could affect the media cache cleaning efficiency. We summarize the system implications of the unique HA-SMR performance characteristics for building large scale storage systems. Furthermore, we replay real-world traces against the HA-SMR drives and explain the results using the knowledge we have obtained from the performance evaluation.

Based on our observations of SMR drive performance and by fully exploiting the HA-SMR model, we propose a *host-controlled indirection buffer* (called *H-Buffer*) which has the potential to improve the performance of HA-SMR drives by combining the strengths of both the drive and the host and transferring SMR-unfriendly I/O traffic patterns into more favorable ones. Without a comprehensive design, we have demonstrated the usage and benefit of the H-Buffer in a simple case study.

The rest for the paper is organized as follows. In Section 2 we introduce the preliminaries of the HA-SMR drives. Section 3 describes the measurement system and experimental setup. Details of the tests and the corresponding results are presented in Section 4. We discuss the system implications of using HA-SMR drives in Section 5. Real-world trace replay results are analyzed in Section 6. The proposed H-Buffer is described in Section 7. Section 8 presents a case study of the H-Buffer. We discuss the related work in Section 9 and conclude the paper in Section 10.

2 PRELIMINARIES

2.1 SMR Technology

In Shingled Magnetic Recording technology, neighboring data tracks are written overlapped with each other in a shingled fashion [5], [6], [7]. As opposed to SMR, the traditional recording technique is therefore referred to as Conventional Magnetic Recording (CMR) [12], [13].

In SMR drives, “guard tracks” are ordinarily inserted after every a few tracks to divide the whole disk space into many *bands*. Therefore, each band can be individually modified without interfering with other bands. Some of the SMR drives also reserve a small portion of the disk space as *media cache* (also called *persistent cache*) which buffers the non-sequential write requests from the host. From time to time the drive will have to migrate the buffered data blocks from the media cache to their intended shingled bands (*media cache cleaning*). To do this, a Shingled Translation Layer (STL) is needed to map the Logical Block Addresses (LBAs) to the current Physical Block Addresses (PBAs) [15], [16], [17], [18].

2.2 Zone Modeling of SMR Drives

T10 ZBC [9] and T13 ZAC [10] standards abstract an SMR drive as a collection of *zones* which are logical

TABLE 1
Disk Models for SMR Drive

	DM	HA	HM
Zoned Block Dev.	No	Yes	Yes
Media Cache	Needed	Needed	Not Needed
Conventional Zone	N/A	Optional	Optional
Seq-write-pref Zone	N/A	Mandatory	Not supported
Seq-write-req Zone	N/A	Not supported	Mandatory

non-overlapping collections of consecutive LBAs. The notion of a zone in the standards largely corresponds to the band concept in the SMR drive implementation. In the rest of this paper, we will use band and zone interchangeably.

There are two types of zones: *conventional zones* and *write pointer zones* which are conceptually mapped to bands consisting of CMR tracks and SMR tracks respectively. A write pointer zone is designed to be written in a log structure. It has an associated *write pointer* which indicates an LBA within the zone where the next write should target. By contrast, conventional zones have no write pointers. Majority of the zones in SMR drives are shingled. So, unless otherwise stated, we use “zones” to refer to “write pointer zones” in the rest of the paper.

2.3 Sequential and Non-Sequential Writes

Applications can issue a write operation to a zone either at the write pointer or not and the corresponding write operation is defined as either *sequential write* or *non-sequential write* correspondingly. Note that this “sequential” is different from the traditional term of sequential I/O (as opposed to random I/O) that depends on the consecutiveness of the written LBA, the inter-arrival time, and the size of data requested [19]. In this paper, we use the term sequential and non-sequential writes to refer to the write requests happening at and not at the write pointer respectively.

There are two types of write pointer zones, namely sequential write *preferred* zones and sequential write *required* zones. In both cases, sequential writes will go to the targeted LBAs directly and the write pointer will be advanced by the size of blocks successfully written. The write pointer is initially placed at the beginning of the zone (zone is empty). If there are continuous sequential writes, the write pointer will advance accordingly until it reaches the end of the zone (zone is full). Sequential write required zone will reject any non-sequential writes. Conversely, sequential write preferred zone accepts non-sequential writes by temporarily buffering the data in the *media cache* and later migrating them back to the intended zones.

2.4 Three SMR Drive Models

There are three models [9], [10] of SMR drives: drive managed (DM), host aware (HA) and host managed (HM), as summarized in Table 1.

A Host Aware SMR drive consists of sequential write preferred zones and will accept both sequential and non-sequential writes with the help of the media cache. A Host Managed SMR drive is made up of sequential write required zones and will reject any non-sequential write requests. Therefore, such drive cannot be used directly in existing

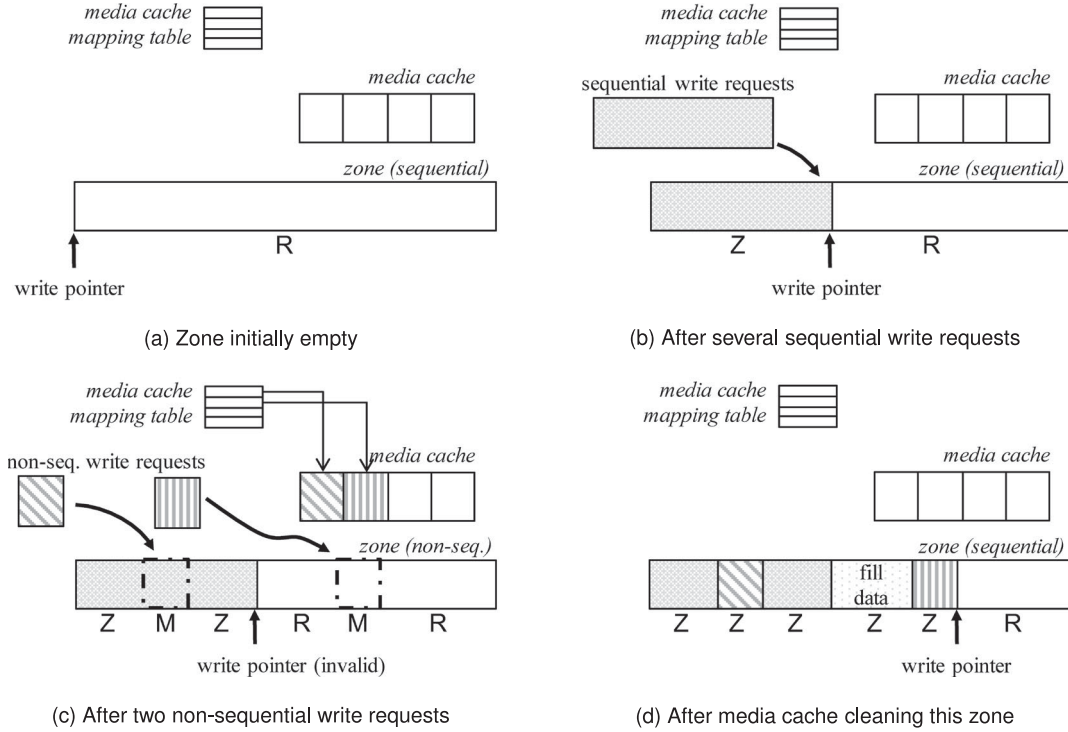


Fig. 1. Zone operations illustrated. (Z, M, R means reading from those LBAs will return data from Zone, Media cache or disk RAM.)

systems without modifying the host software. Both the HA-SMR and HM-SMR drives can optionally have a small number of conventional zones. A Drive Managed SMR drive behaves like a conventional non-shingled hard disk drive with no notion of zones. It exposes a linear space of LBAs to the host hiding all the detailed operations of the media cache and the SMR zones from the host. It can be a “drop-in” replacement solution without any modifications to the existing storage systems.

2.5 Data Handling in HA-SMR Drives

In HA-SMR model, a zone is initially empty with the write pointer pointing to the first LBA (Fig. 1a). Before any non-sequential write, a zone is “sequential”. That is, all the data that have been written, if there is any, are all issued at the current write pointer (Fig. 1b).

A zone is changed to a “non-sequential” state after a non-sequential write request. Such non-sequentially written data is buffered in the media cache and the write pointer is invalidated. The media cache, which is one portion of the disk media, is formatted as a self-describing journal for buffering non-sequentially written data. Subsequent writes to a non-sequential zone are regarded as non-sequential writes and will also be redirected to the media cache (Fig. 1c). Later, a *media cache cleaning* process will migrate all the buffered data back to the targeted zones (Fig. 1d).

A media cache cleaning algorithm will read out the oldest journaled block (i.e., in a FIFO order) from the media cache and any other blocks in the media cache that belong to the same zone. It will also read out the existing data in the corresponding zone. Then all the data blocks will be combined into a consecutive extent and written back to the zone. The zone will then be converted back into a sequential state with its new write pointer properly set. After this, the

algorithm can proceed with the next oldest block in the media cache journal [12], [13].

When performing media cache cleaning, if there is any “gap”—LBAs not written yet by the host—before the new write pointer location, synthesized “fill data” will be generated and written to such gap space (Fig. 1d) [20]. This guarantees that the new write pointer will be the LBA right after the highest LBA written by the host.

For a sequential zone (Figs. 1a, 1b, and 1d), reading data from a location before the write pointer location will retrieve data from the drive which may include the disk resident “fill data” (Fig. 1d), while reading from a location beyond the write pointer will simply return synthesized “fill data” from disk RAM without accessing the platter. For a read request to a non-sequential zone, data can be accessed from either the zone, media cache or disk RAM in different cases as shown in Fig. 1c.

2.6 Zoned Block APIs and Zone Open/Close Semantics

Each zone needs to be opened either *explicitly* or *implicitly* for writing. A zone can be explicitly opened by issuing the OPEN ZONE command [9], [10] or implicitly opened by writing data to the zone. Read operations, however, do not require opening a zone beforehand.

Zone meta-data (write pointer and various flags) that are frequently accessed during read/write operations are kept in the disk RAM. When a write operation issued to the drive updates the zone meta-data in the RAM of the drive, the on-disk copy of the zone meta-data becomes stale and the disk is at risk of losing data. It is critical for the zone meta-data in the drive’s RAM to be frequently synchronized back to the platter. To reduce the overhead of such frequent synchronization of zone meta-data, HA-SMR drives have some limited “open zone resource”

TABLE 2
HA-SMR Drive Specification

Attribute	Value
interface	ATA ZAC
model	host-aware disk model
Seagate model No.	ST8000AS0022-1WL
prototype firmware revision	SN03
capacity	8001.563 GB
cache/buffer size	16384 KB
normal media rotation rate	5,980RPM
zone number/size	29,809 / 256 MB
conventional zones	64 (0-63)
seq-write-pref zones	29,745 (64-29808)
seq-write-req zones	Not Applicable
optimal open zone num	128
optimal non-sequential zone num	8

that can hold the zone meta-data intact through unexpected power losses. Such “open zone resource” is allocated for each opened zone and will be reclaimed when the zone is closed [20].

2.7 Open Zone and Non-Sequential Zone Issues

As shown in the standards, there is a recommended maximum number of open zones which corresponds to the limited open zone resources described in Section 2.6. From system boot-up, the first few open zone operations only result in resource allocation without causing any disk synchronizations. However, as the host opens more and more zones and the open zone resource is used up, some zones have to be closed to release the open zone resource. Such zone close operations are done either explicitly by the host issuing CLOSE ZONE commands or implicitly by the drive selecting and closing zones on its own. Closing a zone will write back the zone meta-data to the drive incurring expensive disk operations. As a result, if the zone working set for write operations consists of a large number of zones, the write performance is expected to suffer because of the “open zone resource” thrashing (*Open Zone Issue*).

Besides, the standards also indicate a recommended maximum number for non-sequential zones. The reason is that the drive has a limited capability to clean non-sequentially written data from the media cache to the intended zones. When the non-sequential write requests are too many and span a large number of zones, media cache resources are depleted, cleaning may become blocking, and thus hurts the performance of both the writes and reads (*Non-sequential Zone Issue*).

3 MEASUREMENT ENVIRONMENT

3.1 SMR Sample Drives and Testing System

We use several Seagate 8TB Host-Aware SMR sample drives for all the tests and performance evaluations (Table 2). The testing system is a Dell PowerEdge R420 1U Server. It is equipped with two Intel(R) Xeon(R) E5-2407 2.20 GHz processors and 32 GB DDR3 DIMM memory.

3.2 Library and Benchmarking Tools

libzbc [21] is a user-level library developed by HGST to manipulate zoned block devices. The sample drives are

compatible with the r04 branch of *libzbc* v4.0.0. In our tests, we use *libzbc* to get the geometry (zone number and size) of the drive, to monitor the write pointers and the count of non-sequential zones, and to reset the zone write pointers when needed. *fio* [22] v2.6 is used to replay various traces to the sample drives and to measure performance in terms of latency, throughput, IOPS, run time, etc.

4 EVALUATION RESULTS

Our evaluation has a special emphasis on the zoned block APIs and the HA-SMR specific media cache effects on the I/O performance. In this section, we present first the testing results of the fundamental SMR drive parameters, then an investigation on the performance impact of the open zone issue and non-sequential zone issue, and finally a study on how different workloads affect the media cache cleaning efficiency. In these tests, write-cache and read-ahead of the drives are disabled to exclude the performance interference of the disk RAM and to isolate the investigation of the zone specific factors.

4.1 Testing for Fundamental SMR Drive Parameters

We carried out tests similar to Skylight [12] to discover the fundamental parameters of the internal structures of the HA-SMR drives, including media cache location and size, mapping table size, as well as the band size, the mapping type, etc. The results are summarized in Table 3.

As can be seen from the result, similarly, our sample drives also do an aggressive and FIFO cleaning. “Aggressive” means that the drive triggers media cache cleaning when the drive becomes idle for a short duration [12]. Conversely, we have found that the band (zone) size is much larger for our HA-SMR sample drives (256 MiB, which is actually the zone size) than that of the two DM-SMR drives described in [12] (15-40 MiB).

Besides, we also measure the average per zone idle cleaning time by monitoring the count of non-sequential zones using *libzbc* [21]. The average per zone idle cleaning time is calculated by dividing the idle cleaning time by the number of non-sequential zones that are cleaned during the idle cleaning time. Here the *idle cleaning time* means the duration of time since the disk becomes idle till the non-sequential zone count hits zero. We find that the cleaning time varies widely under different workload settings (Table 3). We investigate and summarize different factors that can affect the cleaning efficiency in Section 4.4.

4.2 Open Zone Issue

Although the drive specifies the recommended maximum number of the open zones, system designers may need to know exactly how the performance degrades when the recommendation is not followed. To provide a reference point for the system designers and to motivate the solutions, we conduct the following measurements to evaluate the open zone issue for both sequential and non-sequential workloads.

We use a light and bursty workload in this test to exclude the media cache cleaning impact and isolate the open zone issue for analysis. Such workload is characterized by long idle time between bursty I/O requests and can be typically found in personal computers and archival systems.

TABLE 3
Fundamental Parameters Testing Result (Compared with Skylight [12], Our Result Highlighted in Bold)

Drive Model	8TB HA (our sample)	5TB DM [12]	8TB DM [12]
Media cache type	Disk	Disk	Disk
Media cache location	Single, at outer tracks	Single, at outer tracks	Single, at outer tracks
Media cache size	25.6 GB	20 GB	25 GB
Media cache mapping table size	185,000	200,000	250,000
Band size	256 MiB (zone size)	13-36 MiB	15-40 MiB
LBA-PBA mapping	Static	Static	Static
Cleaning type	Aggressive, FIFO	Aggressive, FIFO	Aggressive, FIFO
Cleaning time	1~30+ sec/zone	0.6~1.6 sec/band ^a	0.6~1.6 sec/band ^b

^{a,b} However, the journal version of Skylight [13] came up with a worst-case cleaning time of 45 seconds for a single band.

The test program issues 1,000 write requests in a round-robin fashion among a number of zones during each burst. The number of the open zones ranges from 20 to 200. Both sequential writes and non-sequential writes are evaluated. Besides, we test with large (512 KB) and small (4 KB) I/O request sizes. The results are summarized as boxplot in Fig. 2. The x -axis of Fig. 2 represents the number of open zones and y -axis represents the throughput. The highest and lowest bars in the plot show the maximum and minimum throughputs respectively. The middle rectangular box represents the throughput between the first and the third quartiles. The horizontal line in the middle of each box represents the median throughput. We observe that for sequential writes, the performance drops clearly from 120 to 140 zones, which is consistent with the self-reported 128 recommended maximum open zone number (Table 2). E.g., the median throughput decreases by 57 percent when the number of open zones increases from 120 to 140 in the 4 KB case. We will explore a possible solution for such a performance degradation in Section 8.

Surprisingly, the performance of non-sequential writes shows no significant throughput degradation as the open zone number increases. This is probably because non-sequential writes are redirected to media cache and the self-describing journal structure (see Section 2.4) guarantees data reliability without using too many “open zone resources”.

4.3 Non-Sequential Zone Issue

The recommended maximum non-sequential zone number indicates a suggested number of zones that can be non-sequentially written in order to limit the media cache

cleaning overhead. Nevertheless, legacy zone-unaware workloads could inevitably scatter non-sequential write requests to more zones than the recommended number (8 for our sample drives, Table 2). Therefore, we still want to study the performance characteristics of the legacy workloads.

In the previous test, we find that a light and bursty *non-sequential* workload experiences no noticeable performance degradation when the open zone number goes beyond the recommend maximum. This is because the idle time is long enough for the idle cleaning to finish before next burst of I/O requests arrives (the idle time needed to complete the cleaning depends on the workload, on which we have more discussions in Section 4.4). Therefore, a light and bursty workload cannot show the media cache cleaning impact on I/O performance.

In this test we design a long-run write intensive workload that covers different number of zones. Such workload is able to deplete the media cache resource and forces the drive to do the blocking cleaning while serving write requests. Specifically, a 256 KB-sized non-sequential write workload is performed for 2 hours into 128 and 256 zones respectively. The throughput and the count of non-sequential zones are summarized in Fig. 3.

In both 128 and 256 zone cases of Fig. 3, the non-sequential zone count decreases periodically indicating that the drive starts the blocking cleaning regularly while serving I/O. Based on the throughput and the non-sequential zone count plots, it can be seen that each time the drive starts to clean the media cache, the throughput drops from

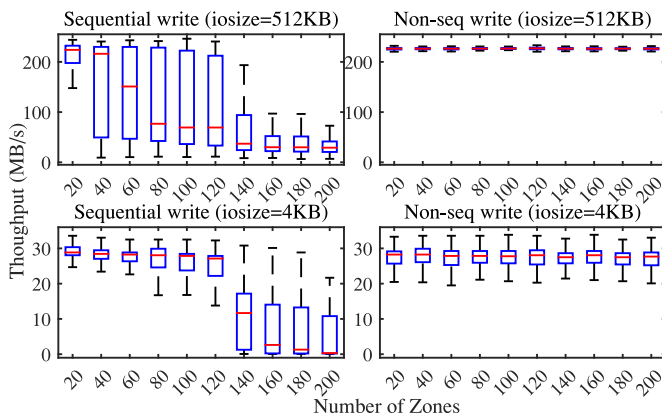


Fig. 2. Throughput versus number of open zones.

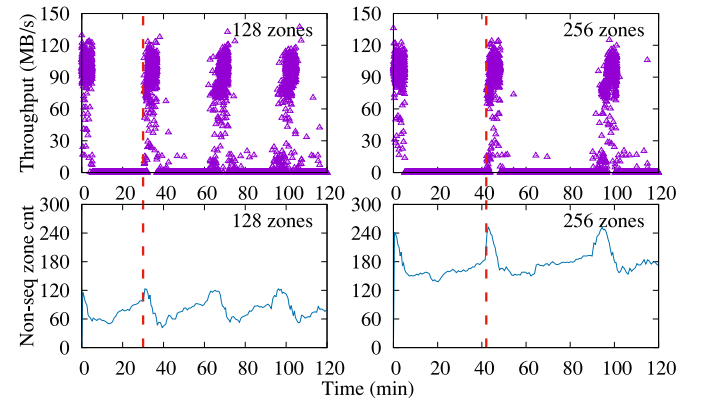


Fig. 3. Throughput and non-sequential zone count versus time when writing to 128 or 256 zones. Vertical dash lines show the correlation between the throughput and the cleaning process.

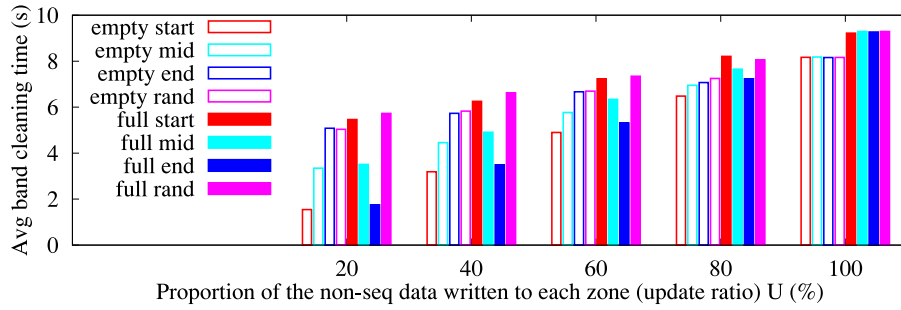


Fig. 4. Impact of update ratio and the I/O request location on the average per zone idle cleaning time.

over 100 MB/s to a very low rate around 0.1 MB/s (0.1 percent of the normal throughput). During the first cleaning cycle, the throughput stays at the low rate for over 25 minutes in the 128-zone case and over 37 minutes in the 256-zone case. This is certainly a very severe degradation of performance that should concern system designers.

We speculate that the low throughput interval will be prolonged when the number of the non-sequential zones increases. The reason is that as the number of non-sequential zones increases, less space is reclaimed in media cache when the drive cleans one targeted zone (migrating media cache buffered data back to this targeted zone by reading, combining, and rewriting this zone). Therefore, more zones need to be cleaned to reclaim the same media cache space enough for the drive to perform I/O in a normal rate. To verify our speculation, we create an extreme workload in which the non-sequential writes span the whole drive (29,808 zones). Consequently, the first blocking cleaning takes so long that the testing program breaks down after about 100 minutes of low-rate interval. A possible explanation is that the cleaning speed is extremely slow and the drive throttles the incoming writes hard for a long duration that causes some timeout faults in the Linux I/O stack. However, such an extreme workload is not expected in a realistic production environment.

4.4 Media Cleaning Efficiency

From the test in Section 4.1, we notice that the average per zone idle cleaning time varies in a wide range (1~30 + s) under different workloads. We hypothesize that the zone cleaning efficiency depends on the workload characteristics including the *updated ratio*, the *I/O request location*, *I/O request size*, the *number of I/O requests*, and the *ordering in which non-sequential data is issued to the drive*. Here the updated ratio (U) of a zone is defined as the proportion of data that is non-sequentially written within a zone. The I/O request location is defined as the location where the update happens within a zone. In this section, the impact of these factors on media cache cleaning efficiency is evaluated.

4.4.1 Update Ratio and the I/O Request Location

We design the first test to study the impact of the update ratio and the I/O request location on the average per zone idle cleaning time. The test program updates 1,000 zones in a round robin fashion with different update ratios ($U=20, 40, 60, 80$, and 100 percent). Note that such update traffic is a non-sequential write workload. Four different I/O request locations are used: the beginning of the zone (start), the middle of

the zone (mid), the end of the zone (end) and random offsets within the zone (rand). The initial state of a zone can be either empty or full. We calculated the average per zone cleaning time (see Section 4.1) and summarize it in Fig. 4.

From the figure, we can have the following observations: As U increases, the cleaning time goes up accordingly since more data blocks need to be migrated from media cache. If we start from an empty zone, when U increases to 100 percent, there is actually no difference whichever LBA location we pick within the zone (i.e., start, mid, end or rand). From the bar plot we see the four cases converge at $U = 100$ percent. The same happens if we start from a full zone.

When writing to an empty zone, the closer the I/O request location is to the zone end, the longer it takes to clean that zone. Note that randomly selected LBAs also result in the same highest cleaning time. The reason is that when migrating data from media cache back to intended zones, the drive needs to write some synthesized fill data to the unwritten areas before the write pointer (Section 2.5). Therefore, updating higher LBA locations in an empty zone results in more extra data to be read/written in the later cleaning process.

By contrast, when writing to a full zone, the closer the I/O request location is to the zone beginning, the longer the cleaning time is. Similarly, a random selected LBA also results in the same highest cleaning time. This is because migrating data into a full zone incurs reading and rewriting that zone and modifying lower LBA will result in more data to be read out and written back.

When $U = 100$ percent, the full zone cases take more time than the empty zone cases do. In fact, identical amount of data is migrated in both cases. It is possibly due to zone meta-data loading overhead. This could also be attributed to some unnecessary reads or some other reasons that are unclear to us.

4.4.2 I/O Request Size and Number of I/O Requests

From the previous test, we know that the average per zone idle cleaning time goes up when more data are written to the zone. However, it is unclear whether the cleaning time will be the same if an equivalent amount of non-sequential data is written with different numbers of I/O requests or different I/O sizes. So we design the next test to investigate the impact of I/O request size and the number of I/O requests.

Instead of increasing U (as defined in Section 4.4.1) by raising the number of I/O requests n , the testing program keeps n constant but increases I/O size s to raise U (left bars in Fig. 5). As the baseline, we still carry out another set of tests by keeping s the same but increasing n . The results are

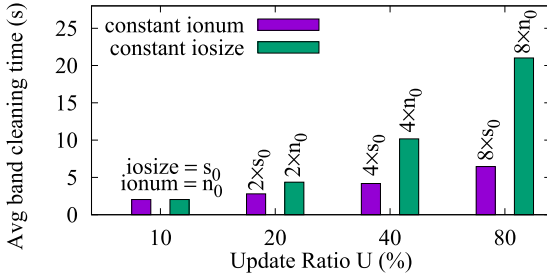


Fig. 5. Compare I/O request size and number of I/O requests.

shown in the right bars in Fig. 5. Note that we start from $U = 10$ percent, $s_0 = 32$ KB, and $n_0 = 800$ in both cases. The zones are initially empty so that the cleaning overhead is close to the effective data movement cost. It can be seen from the figure, in both cases, the cleaning time increases as U grows. But if we increase I/O request size, the cleaning time grows at a slower pace than the case where we raise the number of I/O requests. Therefore, we guess that the number of I/O requests to be cleaned contributes more than the size of the I/O requests does to the cleaning time. We verify this hypothesis using the following test.

In this test, we keep the total amount of data constant, but vary the I/O request size. In other words, if we cut the I/O request size into half, the total number of I/O requests will be doubled. The test program starts from empty zones and updates the first 20 percent of the zones. The I/O request size ranges from 4 to 256 KB. In Fig. 6, we can see that although the same amount of data is migrated, the cleaning time varies a lot. Also we observe that the cleaning time is nearly inverse proportional to the I/O request size when compared to the theoretical inverse proportional plot in the figure. We conclude that the zone cleaning time is affected more by the number of I/O requests than the I/O request size. This probably is because the time for random seeks when migrating data blocks and/or the mapping manipulation (read/update) cost of the non-sequentially written data dominates the cleaning time.

4.4.3 Benefit of Zone-Oriented Reordering

Legacy I/O stack has no knowledge of zone boundary, therefore individual I/O requests may jump back and forth among different zones. We observe from the following experiments that if the host can group random write requests according to their destination zones (namely *zone-oriented reordering*) before sending them into the SMR drive, the cleaning efficiency can be improved.

The program randomly fills the first 0.1 percent of each of 1,024 empty zones with 4 KB I/O size in two different ordering:

- Interleaved (Default): the test program issues a 4 KB non-sequential write to the first zone, then issues another 4 KB write to the second zone, and all the way to the last zone. Then it starts over from the first zone in a round-robin fashion until each of the zones has the first 0.1 percent filled. Here the data blocks from the same zone are interleaved in the media cache.
- Zone-oriented reordered: the test program randomly fills the starting 0.1 percent of the first zone using

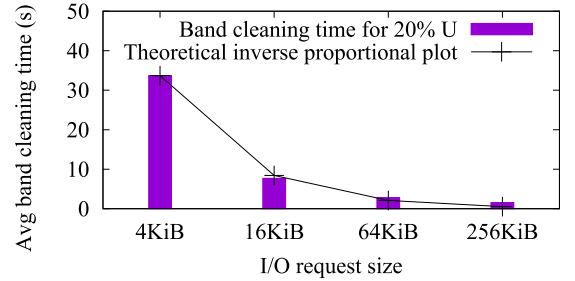


Fig. 6. Average per zone idle cleaning time with the same total amount of data written but in different I/O request sizes.

4 KB data blocks, then the starting 0.1 percent of the second zone, and all the way to the last zone. The data blocks from the same zone are grouped together in the media cache.

In order to help examining the media cache cleaning status later, the program reads all the data blocks out in the order they are written immediately after the completion of the write workload. In Fig. 7, we summarize the latency, the throughput, and the count of non-sequential zones of the default interleaved ordering (Fig. 7a) and the zone-oriented reordering (Fig. 7b).

We notice that during the write workload there are some “early cleaning” activities, shown by blue circles in the non-sequential zone count plot in both Figs. 7a and 7b. Such “early cleaning” happens before the disk becomes idle and does not block the ongoing workload. The reason for such “early cleaning” is that the workload exceeds the *media cache raw size* limit [12], [13], given that we use a low queue depth and a small I/O size. In both ordering, when early cleaning starts, the latency increases and the throughput decreases. Additionally, the distinct phase change in the read throughput plots in Figs. 7a and 7b (marked by black dashed lines) also verifies the existence of the early cleaning.

We observe that zone-oriented reordering reduces the write completion time by 10.6 percent, and the idle cleaning time by 43.0 percent from the default interleaved ordering, indicating a more efficient cleaning both during the workload (early cleaning) and after the workload (idle cleaning). This is because zone-oriented reordering avoids repeated data migration to the zones. For example, as shown in the non-sequential zone count plot in Fig. 7a, the early cleaning in the default interleaved case has almost cleaned all the 1,024 zone once. However, during the idle cleaning, these zones are cleaned again, where migrating new data from the media cache incurs reading and rewriting those old data blocks that was already there since last migration.

5 SYSTEMS IMPLICATIONS

5.1 Sequential Writes versus Non-Sequential Writes

The long-held understanding that sequential writes have better performance than non-sequential writes does not always apply to HA-SMR drives although the definition of sequentiality is somehow different from the traditional ones (Section 2.3). In some cases, we do observe that bursty non-sequential writes could perform better than sequential writes (Fig. 2). Here are several reasons for that:

- Non-sequential writes will be redirected to the media cache which is located in the higher-performance outer tracks (Table 3);

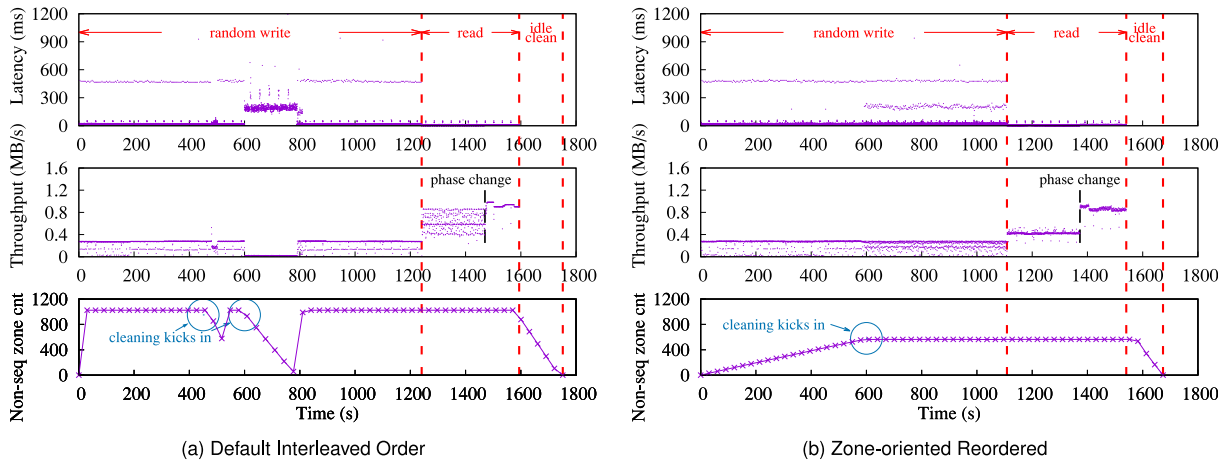


Fig. 7. Zone-oriented reordering improves the media cache cleaning efficiency, reflected by reduced write completion time and idle cleaning time.

- Media cache buffering converts random writes into physically sequential writes;
- Sequential writes could be severely affected by the open zone issue; and
- Light and bursty workloads do not suffer from the performance degradation caused by media cache cleaning as write intensive workloads experiences.

Therefore, for a bursty non-sequential workload that fits within the media cache, the media cache can hide the random cost, so the non-sequential workload may have even a better performance than a sequential workload. However, if the non-sequential workload continues, the application will eventually experience the high cleaning cost when the media cache fills up (Sections 2.7 and 4.3).

5.2 Open Zone Number

Due to the internal limitation of the open zone resource in the HA-SMR drive, a limited number of zones can be open for writing. Frequent switching among sequentially written zones means frequent disk synchronizations for the open zone meta-data, which causes the write performance to drop

sharply, even if the application is performing sequential writes to the zones. Therefore, system designers should also avoid making the working set for sequential writes beyond the recommended maximum number of open zones.

5.3 Non-Sequential Zone Number

The penalty of cleaning non-sequential writes is more severe when more zones are non-sequentially written, in which case the low throughput interval is prolonged. During the cleaning time, the throughput drops three orders of magnitude compared with normal performance. Disk vendors may give a recommended maximum number of non-sequential written zones. However, this number is only a rough estimate considering the overall capability of the drive including the size of the media cache, the media cache map size, the disk's internal processing power, and the available disk RAM supporting the cleaning. If non-sequential writes are inevitable, the application developers for HA-SMR drives should keep the number of non-sequential zones as few as possible to avoid this severe performance degradation. From another perspective, the HA-SMR drives will fit well in a

TABLE 4
Zone Specific Analysis for MSR Traces (Sorted by the Write Request #)

trace	total req #	write req #	total write size (GB)	footprint (#zones)*	Write Ratio (%)	trace	total req #	write req #	total write size (GB)	footprint (#zones)*	Write Ratio (%)
wdev_3	682	671	0.003	5	98.39	usr_0	2,237,889	1,333,406	13.077	34	59.58
wdev_1	1,055	1,055	0.005	4	100.00	src2_0	1,557,814	1,381,085	9.340	38	88.66
rsrch_1	13,780	13,738	0.159	84	99.70	web_0	2,029,945	1,423,458	11.673	35	70.12
src2_1	657,774	14,104	0.180	125	2.14	src1_2	1,907,773	1,423,694	44.147	16	74.63
web_3	31,380	21,330	0.424	14	67.97	ts_0	1,801,734	1,485,042	11.340	47	82.42
hm_1	609,311	28,415	0.541	11	4.66	stg_0	2,030,915	1,722,478	15.090	30	84.81
web_2	5,175,368	38,963	0.784	40	0.75	usr_2	10,570,046	1,994,612	26.469	926	18.87
rsrch_2	207,587	71,223	0.289	72	34.31	src1_1	45,746,222	2,170,271	30.349	381	4.74
web_1	160,891	73,833	0.649	40	45.89	proj_1	23,639,742	2,496,935	25.576	1,475	10.56
proj_4	6,465,639	95,865	1.010	294	1.48	hm_0	3,993,316	2,575,568	20.477	45	64.50
proj_3	2,244,644	116,341	2.627	254	5.18	prn_1	11,233,411	2,769,610	30.785	1,222	24.66
mds_1	1,637,711	116,676	1.540	188	7.12	proj_2	29,266,482	3,624,878	168.686	1,474	12.39
wdev_2	181,266	181,077	1.407	32	99.90	proj_0	4,224,524	3,697,143	144.267	34	87.52
stg_1	2,196,861	796,452	5.986	224	36.25	usr_1	45,283,980	3,857,714	56.127	1,886	8.52
src2_2	1,156,885	805,955	39.282	275	69.67	prn_0	5,585,886	4,983,406	45.965	85	89.21
wdev_0	1,143,261	913,732	7.146	27	79.92	prxy_0	12,518,968	12,135,444	53.797	58	96.94
mds_0	1,211,034	1,067,061	7.365	19	88.11	src1_0	37,415,613	16,302,998	809.159	478	43.57
rsrch_0	1,433,655	1,300,030	10.818	59	90.68	prxy_1	168,638,964	58,224,504	724.816	169	34.53

* Here footprint is defined as the count of zones that are written by the traces.

TABLE 5
MSR Traces Average Latency (Sorted by the Write Request #, see Table 4)

Trace	HDD Avg Lat.(μ s)	SMR Avg Lat.(μ s)	Trace	HDD Avg Lat.(μ s)	SMR Avg Lat.(μ s)	Trace	HDD Avg Lat.(μ s)	SMR Avg Lat.(μ s)	Trace	HDD Avg Lat.(μ s)	SMR Avg Lat.(μ s)
wdev_3	240.92	221.93	proj_4	451.27	593.90	usr_0	1267.99	2300.85	hm_0	2376.67	3213.04
wdev_1	159.72	220.67	proj_3	384.55	853.91	src2_0	612.04	1461.26	prn_1	2206.03	12116.30
rsrch_1	2207.60	8579.06	mds_1	781.69	1547.47	web_0	2071.97	2489.80	proj_2	1191.22	2248.36
src2_1	743.30	977.02	wdev_2	490.66	718.95	src1_2	978.80	4445.12	proj_0	876.53	1480.60
web_3	2400.37	1555.28	stg_1	848.13	2683.33	ts_0	536.96	1068.41	usr_1	1415.70	4703.44
hm_1	436.98	785.13	src2_2	2836.14	3662.42	stg_0	843.51	2588.73	prn_0	2013.20	4347.09
web_2	344.52	406.65	wdev_0	442.32	920.41	usr_2	1781.26	4209.63	prxy_0	1025.67	592.48
rsrch_2	1321.52	951.67	mds_0	629.77	827.67	src1_1	729.98	1376.58	src1_0	1490.85	3007.77
web_1	1478.16	1050.01	rsrch_0	423.46	2501.85	proj_1	1471.86	4220.74	prxy_1	2943.32	3259.39

hierarchical storage architecture as the second tier storage where the first tier—possibly flash—can filter out most of the requests of non-sequential writes. It may also work well as the primary tier if the can reduce non-sequential writes and/or improve the cleaning efficiency.

5.4 Media Cache Cleaning Efficiency

Generally, the cleaning time is positively correlated with the total amount of data to be rewritten and the number of buffered write requests to be migrated. Between the two, the number of requests contributes more to the cleaning time. This will give us a hint that larger non-sequential I/O size is favorable in a sense that the cleaning time will be shorter because it results in fewer non-sequential write requests logged in the media cache.

Zone-oriented reordering will potentially increase the cleaning efficiency and reduce the workload finishing time. However, individual request may experience a longer queuing delay since the host has to accumulated some number of I/O requests in the queue to perform the reordering. Therefore, such reordering technique will be more suitable for batching I/Os where the overall completion time is more critical than individual I/O latency.

6 REPLAY OF MICROSOFT RESEARCH CAMBRIDGE TRACES

Previous tests mainly focus on investigating the drive's zone-specific performance using synthesized traces. We are also interested in the I/O performance of HA-SMR drives under typical real-life workloads. Besides, we can apply the knowledge obtained from the previous performance tests (Section 5) to help predicting and analyzing the real-world trace replay results.

We replay the 36 one-week-long Microsoft Research Cambridge (MSR) traces [23] which are collected from enterprise data center workloads. Traces are replayed against both the HA-SMR drives and baseline HDDs (CMR). The HDD model is Seagate ST6000NM0034 (6TB SAS HDD, 7.2 K RPM). We replay the MSR trace using *fiio* with the *sync* engine. Different from the previous settings, write-cache and read-ahead in this test are turned on in both HDDs and SMR drives to mimic realistic production environment. Due to similar reasons, the queue depth and read-ahead sector number are both set to the maximum. Moreover, as reading from a location beyond the write pointer will return synthesized fill data from the SMR drive's RAM and result in unrealistic high throughput (Section 2.5), we "warm up" the SMR drive

before replaying by making the zones full such that the read operations will get fill data from the disk. Our current tests simply "blast" all the I/O requests to the drive without referring to their time stamps.

6.1 Zone-Specific Analysis for MSR Traces

Before the trace replay, the following key statistics are analyzed for the MSR traces: total number of operations, number of write operations, total write size, zone footprint of the writes, ratio of write operations (Table 4).

Those statistics are selected for the following reasons. The number of write operations and the total write amount have direct impact on the media cache cleaning efficiency (Section 2.7). As the write operations of legacy zone-unaware workloads are almost non-sequential (almost never happening at the write pointers), the number of write operations can be regarded as the media cache mapping entries required by the trace. Similarly, the total write amount is a good estimate for the media cache capacity requested by the trace. If the requested resources exceed the drive's media cache capacity (Table 3), cleaning will be triggered, affecting both read and write performances (Section 2.7). The total number of operations and the write operation ratio reflect how intensive the workload is injecting write requests into the SMR drive. Zone footprint for write operations will affect the media cache cleaning efficiency. The more zones the non-sequential writes span, the less efficient the cleaning process is (Section 4.3).

From the statistics, we find that it is the media cache mapping size, rather than the media cache size that becomes the bottleneck to trigger the media cache cleaning. While all the traces are replayed and analyzed (Table 5), four representative ones are especially studied who represent different media cleaning severity, i.e., *proj_4*, *stg_1*, *src1_2*, and *prn_1* (high-lighted in bold in Table 4).

6.2 Representative Traces Analysis

Trace *proj_4* has a small number of write operations and a small total write amount, so we hypothesize that it will not trigger media cache cleaning even if write operations span as many as 294 zones. By contrast, the other three selected traces are expected to trigger media cache cleaning as they have more non-sequential writes than the media cache mapping table size. According to Section 4.3, we hypothesize that media cache cleaning caused performance degradation will be positively related to the zone footprint. So we expect the severity of media cache cleaning impact to be *proj_4* < *src1_2* < *stg_1* < *prn_1*.

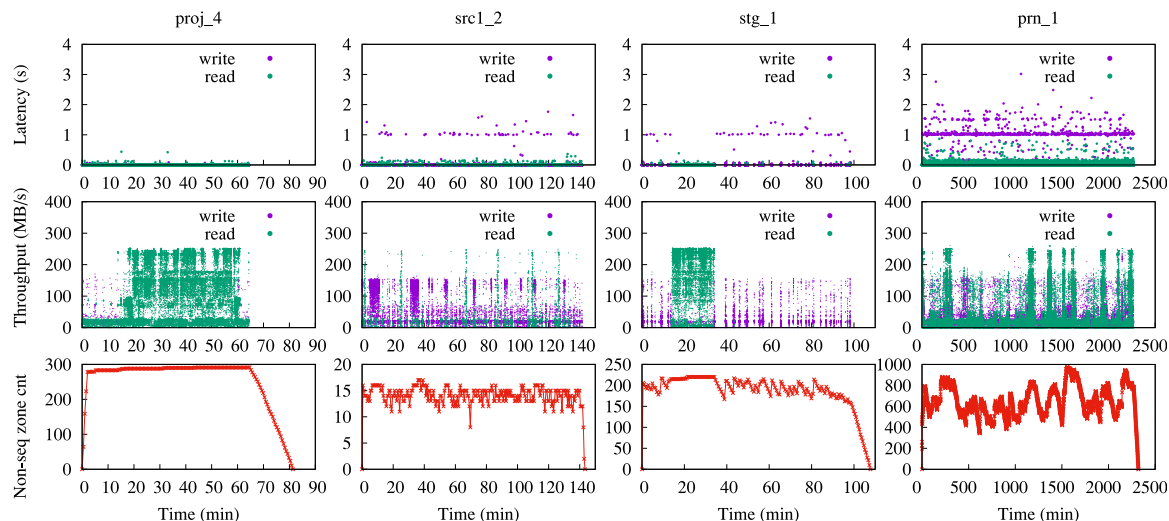


Fig. 8. Replay results of selected traces.

Fig. 8 shows the latency, bandwidth and the non-sequential zone count plots of the selected traces. It can be observed from the non-sequential zone count plot that *proj_4* never triggers media cache cleaning during the workload, while the others do. Note that even having a total write amount as small as 5.99 GB, *stg_1* still triggers cleaning.

We can see from the performance plots of *stg_1* that each time the drive starts media cache cleaning, the throughput decreases and the latency increases. During the almost read-only interval, the non-sequential zone count stays constant. This is because there are fewer write operations being injected into the media cache and the drive is still busy such that the idle cleaning never gets a chance to happen.

Comparing *stg_1* and *src1_2*, we find that *src1_2* spans much less number of zones (16) than that of *stg_1* (224). Therefore, *src1_2* will have less performance degradation even if it imposes more demand for the media cache capacity and mapping table. *prn_1* has the largest zone footprint (1222), implying a serious media cache cleaning impact.

The average write latencies of *proj_4*, *src1_2*, *stg_1*, and *prn_1* are 712.55, 4335.34, 5708.74, and 28426.5 μ s respectively, indicating an increasing degree of performance degradation.

As shown in the throughput plots of *proj_4* and *stg_1*, some read throughput numbers are as high as 250 MB/s. Such high throughput is due to large read request sizes. In *proj_4*, 99.3 percent of the read requests with high throughput (> 100 MB/s) has an I/O size ≥ 32 KB. In *stg_1*, such ratio is 99.8 percent. Although some individual I/Os have high throughput, the average throughput is not as much. The average throughput of *proj_4* is 92.1 MB/s, where the read requests with large I/O size (≥ 32 KB) take up 32.9 percent in all the read requests. The average throughput of *stg_1* is higher (190.6 MB/s), because the large read I/O requests take up a higher ratio (92.6 percent) among total read requests.

6.3 Latency Analysis

We summarize the average latency of all the traces in Table 5. We also collect the 1st, 10th, 50th (median), 90th, and 99th percentile latency numbers, and plot the percentile latency of HDDs against those of the SMR drives, i.e., a sampled Q-Q plot. In Fig. 9, we show the Q-Q plot of the same set of

representative traces as Section 6.2. The most bottom left dot represents the 1st percentile (P01), the second means the 10th latency (P10), so on and so forth. The last point (most top right) represents the 99th percentile latency (P99). The dashed line across the figure shows the boundary where the SMR drive latency is equal to the HDD latency such that any dot to the top left of the dashed line means the SMR drive has a smaller percentile latency, and vice versa.

The Q-Q plot of write requests (Fig. 9a) has greater deviation from the diagonal dashed line than that of the read requests (Fig. 9b). The Q-Q plot for both the I/O requests (Fig. 9c) is a weighted combination of that of the write and read requests.

For write operations (Fig. 9a), traces have comparable 1st and 10th percentile latencies between HDDs and SMR drives, and SMR drives have a better 50th and 90th percentile latencies. This is due to the media cache buffering effect which converts non-sequential writes into physically sequential ones such that most of the SMR write operations have a reduced write latency. However, for SMR drives some 99th percentile write latencies go up by over 2 orders of magnitude from the 90th and this phenomenon does not exist in HDDs. It means 1 out of every 100 write operations may experience a latency in seconds, which is the price to pay when cleaning the media cache. This implies legacy applications will experience more serious performance jitters in write operations. Nevertheless, the 99th SMR write percentile latencies of other traces stay above the dashed line indicating a consistent advantage of HA-SMR drives over HDDs. Such traces never trigger blocking cleaning (e.g., *proj_4*) or only have a moderate blocking cleaning happening (e.g., *stg_1*) as its non-sequential writes span a smaller number of zones.

For read operations, SMR drives has similar 10th, 50th and 90th latency distribution to that of HDDs as the shingled constraint has less impact on read operations. Some traces, e.g., *prn_0*, *prn_1* (show in Fig. 9b), *proj_0*, *proj_1*, *usr_0*, *usr_1*, *usr_2*, experience a longer 99th read latency. We believe the reason could be the media cache cleaning as they coincide with the long 99th percentile of the corresponding write latencies. It may be due to the non-sequential write requests reducing the read-ahead benefits.

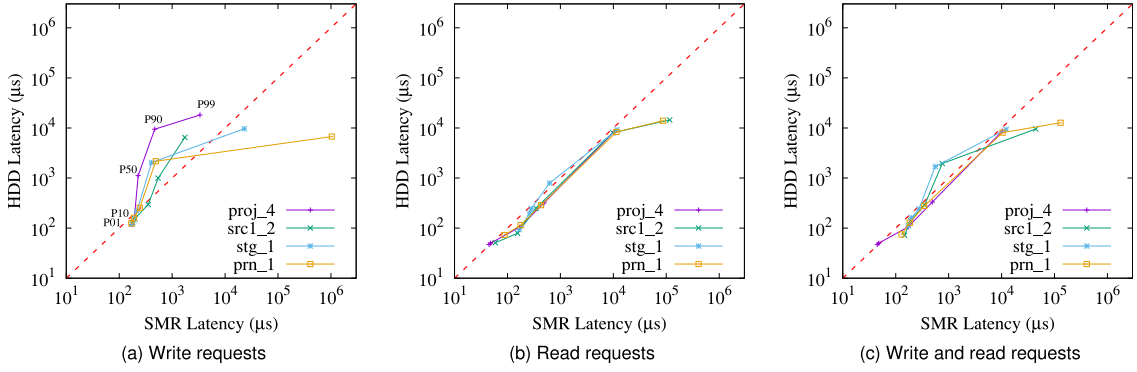


Fig. 9. Sampled Q-Q plot between HDD and SMR drive percentile latency numbers for selected MSR traces. (Note that the plots are in double log scale.)

6.4 Discussion

Note that the baseline HDDs have a slightly higher RPM than that of SMR drives so the comparisons will have some bias in favor of HDDs. Moreover, in a realistic setting, the I/O commands will come to the drive intermittently such that there may be idle time for the drive to clean the media cache without affecting the performance.

7 H-BUFFER

The special performance characteristics especially the performance degradation observed in Sections 4.2 and 4.3 raises quite a challenge for software developers using the HA-SMR drives in a large storage system. To confront this challenge, by exploiting the unique HA-SMR hardware features and host interacting model, we propose a *Host-controlled indirect Buffer* (H-Buffer). H-Buffer embodies an SMR three-data-path system (the *mechanism*) (Fig. 10) which can support a broad spectrum of workload switching algorithms (the *policies*). We believe such separation between mechanism and policy can potentially lead to various remedies to the performance degradation problem due to undesirable workloads. In this section, without a comprehensive design, we conceptually describe H-Buffer as well as its benefit.

7.1 H-Buffer Description

A small number of the zones either shingled or conventional with a user pre-defined or dynamically adjustable size of the drive are reserved as H-Buffer to which incoming data can be redirected by the host. An LBA-PBA mapping table that corresponds to the redirection is maintained in host memory while redirected data blocks are logged as one or many self-describing journals similar to the media cache for mapping recovery. Besides, the host (not the drive) is responsible for migrating buffered data from H-Buffer to the targeted zones (H-Buffer cleaning). Practically, this H-Buffer can be considered as a host-controlled media cache.

7.2 Three Data Paths for HA-SMR Drives

H-Buffer embodies a three-data-path mechanism for SMR drives. As shown in Fig. 10, an HA-SMR drive natively supports data path ① and ②. ① the *direct data path*, represents sequential data being directly written to shingled zones; and ② the *media cache data path*, denotes non-sequential data being buffered into the media cache and later migrated to the intended zones (Section 2.5). Our H-Buffer completes the picture by creating a new data path (③ the *H-Buffer data path*), which is similar to the media cache data path but is controlled by the host instead of the drive.

The three data paths each has its own strength: 1) The direct path ① can accept sequential data without any redirection/migration overhead. 2) Media cache data path ② can handle non-sequential data cleaning without transferring data back and forth with the host, thus has a higher data migrating bandwidth and consumes neither any extra computational resource from that host nor any additional I/O bandwidth with the host. 3) H-Buffer data path ③ can support enhanced cleaning algorithm leveraging the host's bigger memory and more powerful processors such that it can be expected to do a better job on data cleaning. In addition, H-Buffer has the ability to redirect sequential writes too, given that by default HA-SMR drive cannot redirect a sequential workload via ② (Section 8 will show when such redirection for sequential I/O workload is beneficial).

From an SMR's perspective, Host Aware is the *only* model that can exploit the advantage of all three data paths, in that it is the only one which can both handle non-sequential writes and expose zone state information to enable accurate host control. By Contrast, the HM-SMR model rejects any non-sequential write operations, hence cannot support data path ② while the DM-SMR model hides all internal zone state information which precludes the possibility of data path ③

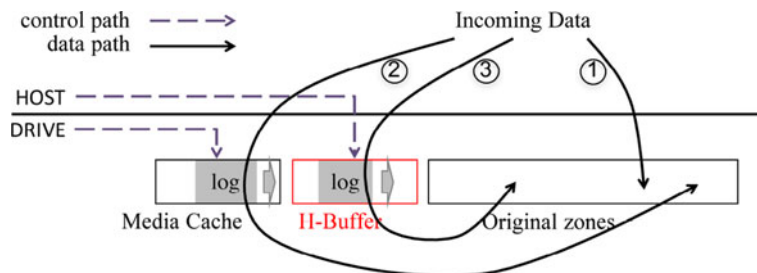


Fig. 10. H-Buffer and the three data paths for HA-SMR drive.

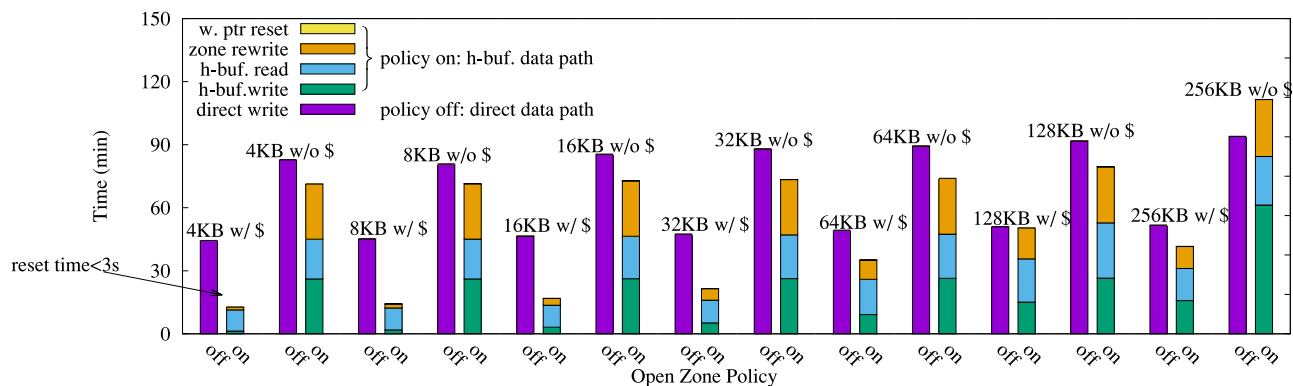


Fig. 11. H-Buffer helps addressing the open zone issue. (“w/ \$” means with write-cache and read-ahead enabled, and vice versa.)

7.3 Data Management Separation Between the Host and the Drive

The three-data-path mechanism enables a separation of data management between the host and the drive. Data path ② has the managing logic in the drive, while ③ has the controlling intelligence in the host. This three-data-path model separates the handling of non-sequential writes and even sequential writes between the host and the drive. Besides, the policy engine switching data requests among the three data paths is also part of the host controlling logic. Therefore, system designers are enabled to explore an optimized combination of management between the host and the drive.

A host-side management can alleviate the media cache cleaning overhead by sharing the data handling task. Actually, the host may potentially do a better job because it has more computational power and RAM size so that it can support more sophisticated buffering and cleaning algorithms. In addition, a host-side management has the advantages of having the knowledge of the workload characteristics, e.g., I/O sequentiality, hot-cold distribution, existence of concurrent streams, and so on. On the other hand, drive-side management does not consume host computational resources or any external I/O bandwidth and can leverage the disk internal bandwidth to do high speed cleaning. In all, H-Buffer and the three-data-path system essentially open a big design space for the policies switching the requests among the three data paths and the separation of management between the host and the drive in order to combine all the advantages under various workloads.

Note that the CPU and memory usage of H-Buffer depends on the complexity of the traffic switching and data migrating policy. So, system designers may have to take the availability of the host’s computational resources into consideration when designing the H-Buffer policies.

8 CASE STUDY: SOLVING OPEN ZONE ISSUE WITH H-BUFFER

In this section, we try to demonstrate the potential of H-Buffer by investigating how it can be used to deal with one of the unique features of HA-SMR drive—the open zone issue. We find that there is a clear performance drop if the number of open zones exceeds the recommended maximum number (Section 4.2).

Applications should always respect this recommended maximum number of 128 sequential zones if they do not want

to hurt the performance by the open zone thrashing phenomenon. However, if some applications do have a requirement to sequentially write over 128 zones simultaneously, the H-Buffer can be used to absorb the incoming writes and later migrate the data back to the intended zones.

We demonstrate H-Buffer’s potential by designing a simple workload switching policy (named: *open-zone-policy*) which addresses the open zone issue (Section 4.2). The open-zone-policy detects concurrent sequential write streams (each stream corresponds to sequential writes to a single open zone) from the workload. If there are more streams than the recommended maximum number of open zones, the policy will switch the sequential write streams into the H-Buffer to keep the number of sequential open zones no more than 128.

To evaluate the open-zone-policy, we create a micro-benchmark workload that contains 200 concurrent sequential write streams. We assume that system reserves 100 zones as H-Buffer. We compare the total run time with the open-zone-policy switched on and off:

- policy switched off (direct data path): 1,000 write operations are directly issued to 200 zones.
- policy switched on (h-buf. data path): 1,000 write operations are initially redirected to the H-Buffer (25.6 G, comprised of 100 zones and comparable to the size of the media cache). Then the host performs the H-Buffer cleaning: read data out from the H-Buffer, write them to the intended 200 zones, and finally reset the write pointers for the 100 H-Buffer zones.

We perform the test with the I/O sizes ranging from 4 to 256 KB. Besides, for each size we experiment with the drive’s write-cache and read-ahead enabled and disabled. We compare the total run time with open-zone-policy turned on and off (Fig. 11). When open-zone-policy is on, the time is broken down into H-Buffer redirected write, cleaning read, zone rewrite and write pointers reset.

We find that resetting 100 zones only takes at most 3 seconds, which is negligible compared with the data transmission time. In small I/O sizes, the H-Buffer data path has shorter total time than that of direct write. As the I/O size increases, the total run time of the direct write grows slowly. This is because when directly writing the 200 streams into the disk, the total run time is dominated by the latency due the open-zone thrashing problem. In comparison, the total time of the H-Buffer path grows faster as the I/O size increases, and surpasses that of direct write in larger I/O

sizes. This means that for larger I/O sizes, the value of the policy in eliminating open-zone thrashing is negated by the overhead of extra H-Buffer migration.

Also, we observe a greater advantage for the open-zone-policy when write-cache and read-ahead are enabled in the drive. The structural reason is unclear to us but we believe that it may be because open-zone thrashing somehow reduces the benefit of the write-cache function. In all, the H-Buffer will bring performance benefit and the benefit is more significant when write-cache and read-ahead are enabled and the I/O request sizes are smaller.

In conclusion, even though the open-zone-policy does extra I/O operations in the H-Buffer data path, under certain workload it still takes less total time than that of directly writing to a large number of zones. In this case study, we simply switch all the I/O requests to the H-Buffer data path. However, practical policies can be more sophisticated by switching each individual write operation differently into the corresponding optimal data path.

9 RELATED WORK

9.1 SMR Drive Characterization/Evaluation

Black et al. [24] study the particular SMR characteristics in large-scale cold-data archival storage systems in which the rack-level power provision requires the SMR drives to be frequently spun up and down. Aghayev and Desnoyers [12], [13] use both software and hardware approaches for DM-SMR drive reverse engineering. Different from existing works, our work focuses on microscopic performance characteristics for the HA-SMR drives with a special emphasis on the SMR zone specific APIs.

9.2 SMR Data Handling Solutions

Amer et al. explore the design space for the data management and layout of SMR drives [25], [26]. Cassuto et al. [15] propose two *indirection systems* for SMR drives, i.e. *set-associative STL* and *S-block STL*.

H-SWD [27], [28] adopts a circular log layout and implements hot/cold data identification for data placement decision. Hall et al. [16] propose a data handling algorithm which divides SMR drives into two shingled write regions, namely *I-region* (larger) and *E-region* (smaller). Kadekodi et al. [29] explore two models, namely Strict-Append and Caveat-Scriptor, for HM-SMR operations. He and Du exploit static [17] and dynamic [18] track level mapping schemes to reduce the write amplification and garbage collection overhead. Manzanares et al. design a zone-based extent allocator (ZEA) [30] that only performs sequential allocation for data and meta-data on SMR drives to conform to the shingled constraint.

SFS [31] uses the full knowledge of the disk layouts and allows more efficient optimization compared to a disk firmware based approach. Jin et al. propose a file system called HiSMRfs [32] which separates the meta-data and data space into SSD and SMR drives. Aghayev et al. [33] introduce *ext-lazy* which reduces the meta-data block writes in Ext4 to reduce the performance overhead due to media cache cleaning.

Our work differs from other SMR data handling solutions in that we focus on the software solution for HA-SMR model. We propose the three-data-path system

(enabled by H-Buffer) for HA-SMR drives by exploiting the unique Host Aware model.

10 CONCLUSIONS AND FUTURE WORK

We present a study on Host Aware SMR drives by carrying out in-depth performance evaluations on several HA-SMR sample drives. Specifically, we investigate the drive internals, the performance impact of the zoned block APIs and the factors that affect the media cache cleaning efficiency. Based on the empirical observations, we summarize the system implications and utilize the knowledge obtained from the evaluation to analyze the replay results of real world traces. Besides, we propose a novel host-controlled indirection buffer for improving the performance in HA-SMR drive systems. A case study of the open zone issue shows the potential of such host-controlled buffer.

In the future, we plan to further investigate the media cache cleaning mechanism and explore more production workloads for testing. Moreover, we are going to comprehensively investigate the data path switching policies when using H-buffer to fit various workloads such that HA-SMR drives can be used to construct large scale storage systems that support various types of workloads.

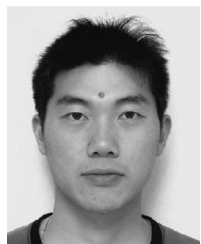
ACKNOWLEDGMENTS

This work is partially supported by NSF I/UCRC Center for Research in Intelligent Storage (CRIS) and the US National Science Foundation under awards: 1525617, 1439622, 1305237, and 1421913.

REFERENCES

- [1] S. Piramanayagam, "Perpendicular recording media for hard disk drives," *J. Appl. Physics*, vol. 102, no. 1, 2007, Art. no. 011301.
- [2] R. E. Rottmayer, et al., "Heat-assisted magnetic recording," *IEEE Trans. Magn.*, vol. 42, no. 10, pp. 2417–2421, Oct. 2006.
- [3] M. H. Kryder, et al., "Heat assisted magnetic recording," *Proc. IEEE*, vol. 96, no. 11, pp. 1810–1835, Nov. 2008.
- [4] R. L. White, R. Newt, and R. F. W. Pease, "Patterned media: A viable route to 50 Gbit/in² and up for magnetic recording?" *IEEE Trans. Magn.*, vol. 33, no. 1, pp. 990–995, Jan. 1997.
- [5] R. Wood, M. Williams, A. Kavcic, and J. Miles, "The feasibility of magnetic recording at 10 terabits per square inch on conventional media," *IEEE Trans. Magn.*, vol. 45, no. 2, pp. 917–923, Feb. 2009.
- [6] I. Tagawa and M. Williams, "High density data-storage using shingle-write," presented at the FA-02, International Conference on Magnetics, Sacramento, CA, May 2009.
- [7] G. Gibson and M. Polte, "Directions for shingled-write and two-dimensional magnetic recording system architectures: Synergies with solid-state disks," Parallel Data Lab, Carnegie Mellon Univ., Pittsburgh, PA, USA, Tech. Rep. CMU-PDL-09-014, 2009.
- [8] Seagate Technology, "Affordable active archive hard drives for cloud storage," (2014). [Online]. Available: <http://www.seagate.com/enterprise-storage/hard-disk-drives/archive-hdd/>
- [9] INCITS T10 Technical Committee, "Information technology-zoned block commands (ZBC)," Draft Standard T10/BSR INCITS 536, American National Standard Institute, Inc., Dec. 2015. [Online]. Available: <http://www.t10.org/drafts.htm>
- [10] INCITS T13 Technical Committee, "Zoned-device ATA command set (ZAC) working draft," (2016). [Online]. Available: <http://www.t13.org/Documents/MinutesDefault.aspx?DocumentType=4&DocumentStage=2>
- [11] T. Feldman and G. Gibson, "Shingled magnetic recording areal density increase requires new data management," *USENIX Login*, vol. 38, no. 3, pp. 22–30, 2013.
- [12] A. Aghayev and P. Desnoyers, "Skylight—a window on shingled disk operation," in *Proc. 13th USENIX Conf. File Storage Technol.*, 2015, pp. 135–149.

- [13] A. Aghayev, M. Shafaei, and P. Desnoyers, "Skylight—a window on shingled disk operation," *ACM Trans. Storage*, vol. 11, no. 4, pp. 16:1–16:28, Oct. 2015. [Online]. Available: <http://doi.acm.org/10.1145/2821511>
- [14] T. R. Feldman, "Host aware SMR," Nov. 2014. [Online]. Available: http://open-zfs.org/w/images/2/2a/Host-Aware_SMR-Tim_Feldman.pdf
- [15] Y. Cassuto, M. A. Sanvido, C. Guyot, D. R. Hall, and Z. Z. Bandic, "Indirection systems for shingled-recording disk drives," in *Proc. 26th IEEE Symp. Mass Storage Syst. Technol.*, 2010, pp. 1–14.
- [16] D. Hall, J. H. Marcos, and J. D. Coker, "Data handling algorithms for autonomous shingled magnetic recording HDDs," *IEEE Trans. Magn.*, vol. 48, no. 5, pp. 1777–1781, May 2012.
- [17] W. He and D. H. Du, "Novel address mappings for shingled write disks," in *Proc. 6th USENIX Workshop Hot Topics Storage File Syst.*, 2014, pp. 5–5.
- [18] W. He and D. H. Du, "SMaRT: An approach to shingled magnetic recording translation," in *Proc. 15th USENIX Conf. File Storage Technol.*, 2017, pp. 121–134.
- [19] C. Li, P. Shilane, F. Douglass, D. Sawyer, and H. Shim, "Assert (!defined (sequential I/O))," in *Proc. 6th USENIX Workshop Hot Topics Storage File Syst.*, 2014, pp. 10–10.
- [20] T. R. Feldman, Personal communication, Jan. 2016.
- [21] HGST, "libzbc," (2016). [Online]. Available: <https://github.com/hgst/libzbc>
- [22] J. Axboe, "Flexible I/O tester," (2016). [Online]. Available: <https://github.com/axboe/fio>
- [23] D. Narayanan, A. Donnelly, and A. Rowstron, "Write off-loading: Practical power management for enterprise storage," *ACM Trans. Storage*, vol. 4, no. 3, 2008, Art. no. 10.
- [24] R. Black, A. Donnelly, D. Harper, A. Ogus, and A. Rowstron, "Feeding the pelican: Using archival hard drives for cold storage racks," in *Proc. 8th USENIX Workshop Hot Topics Storage File Syst.*, 2016, pp. 21–25.
- [25] A. Amer, D. D. Long, E. L. Miller, J.-F. Paris, and S. Schwarz, "Design issues for a shingled write disk system," in *Proc. 26th IEEE Symp. Mass Storage Syst. Technol.*, 2010, pp. 1–12.
- [26] A. Amer, J. Holliday, D. D. Long, E. L. Miller, J. Paris, and T. Schwarz, "Data management and layout for shingled magnetic recording," *IEEE Trans. Magn.*, vol. 47, no. 10, pp. 3691–3697, Oct. 2011.
- [27] C.-I. Lin, D. Park, W. He, and D. H. Du, "H-SWD: Incorporating hot data identification into shingled write disks," in *Proc. 20th IEEE Int. Symp. Model. Anal. Simul. Comput. Telecommun. Syst.*, 2012, pp. 321–330.
- [28] D. Park, C.-I. Lin, and D. H. Du, "H-SWD: A novel shingled write disk scheme based on hot and cold data identification," Poster presented at 10th USENIX Conference on File and Storage Technologies (FAST'12), 2012.
- [29] S. Kadekodi, S. Pimpale, and G. A. Gibson, "Caveat-scriptor: Write anywhere shingled disks," in *Proc. 7th USENIX Workshop Hot Topics Storage File Syst.*, 2015, pp. 16–16.
- [30] A. Manzanares, N. Watkins, C. Guyot, D. LeMoal, C. Maltzahn, and Z. Bandic, "ZEA, a data management approach for SMR," in *Proc. 8th USENIX Workshop Hot Topics Storage File Syst.*, 2016, pp. 26–30.
- [31] D. Le Moal, Z. Bandic, and C. Guyot, "Shingled file system host-side management of shingled magnetic recording disks," in *Proc. IEEE Int. Conf. Consum. Electron.*, 2012, pp. 425–426.
- [32] C. Jin, W.-Y. Xi, Z.-Y. Ching, F. Huo, and C.-T. Lim, "HiSMRfs: A high performance file system for shingled storage array," in *Proc. 30th Int. Symp. Mass Storage Syst. Technol.*, Jun. 2014, pp. 1–6.
- [33] A. Aghayev, T. Ts'o, G. Gibson, and P. Desnoyers, "Evolving Ext4 for shingled disks," in *Proc. 15th USENIX Conf. File Storage Technol.*, 2017, pp. 105–120. [Online]. Available: <https://www.usenix.org/conference/fast17/technical-sessions/presentation/aghayev>



Fenggang Wu received the BS and MS degrees in computer science and technology from Shanghai Jiao Tong University, Shanghai, China, in 2010 and 2013, respectively. He is working toward the PhD degree in computer science in the Department of Computer Science and Engineering, University of Minnesota, Twin Cities. His research focuses on the SMR drives performance characterization and application design, non-volatile memory based storage system, data deduplication, and key-value store.



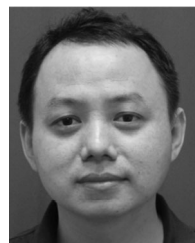
Ziqi Fan received the BE degree from the School of Software, Dalian University of Technology, China, in 2012. He is working toward the PhD degree in computer science and engineering at the University of Minnesota-Twin Cities. He is a member of Center for Research in Intelligent Storage (CRIS) group under the advice of Professor David H.C. Du. His research interests focus on buffer cache policies, SSD and non-volatile memories, file systems and cloud storage, and data deduplication.



Ming-Chang Yang received the BS degree from the Department of Computer Science, National Chiao-Tung University, Hsinchu, Taiwan, in 2010, and the master's and PhD degrees, under the supervision of Prof. Tei-Wei Kuo, in Graduate Institute of Networking and Multimedia from National Taiwan University, Taipei, Taiwan, in 2012 and 2016, respectively. From Aug. 2015 to July 2016, he was a visiting scholar in the Department of Computer Science and Engineering, University of Minnesota, Twin Cities. From Jan. 2017 to July 2017, he was a postdoctoral research fellow in the Institute of Information Science (IIS), Academia Sinica, Taipei, Taiwan. Now, he is an assistant professor in the Department of Computer Science and Engineering, Chinese University of Hong Kong. His primary research interests including emerging non-volatile memory and storage technologies, memory and storage systems, and the next-generation memory/storage architecture designs.



Baoquan Zhang received the BE and MS degrees in computer science from Harbin Engineering University. Currently, he is working toward the PhD degree in computer science and working on storage systems at the University of Minnesota-Twin Cities.



Xiongzi Ge received the PhD degree in computer science from University of Minnesota, in 2017 and another PhD degree in computer architecture from Huazhong University of Science and Technology, in 2012. Since 2016, He has been a full-time researcher in the Advanced Technology Group in NetApp, Inc. His major research includes big data processing, cloud computing and data storage systems.



David H.C. Du received the BS degree from National Tsing-Hua University, Taiwan, in 1974, and the MS and PhD degrees from the University of Washington, Seattle, in 1980 and 1981, respectively. He is currently the qwest chair professor in computer science and engineering at the University of Minnesota-Twin Cities and the director of the NSF Center for Research in Intelligent Storage (CRIS). His current research focuses on intelligent and large storage systems, cyber-physical systems, and vehicular/sensor

networks. He is serves on editorial boards of several international journals. He was a program director (IPA) at National Science Foundation (NSF) CISE/CNS Division from 2006 to 2008. He has served as Conference Chair, Program Committee Chair, and General Chair for several major conferences in database, security and parallel processing. He is fellow of the IEEE (since 1998).

► For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/publications/dlib.